



Atividade 5 - Práticas de Gerência de Configuração - Scripts de construção de software

Objetivo

Implementar um processo automatizado de build que:

- execute o código
- valide com testes
- verifique qualidade
- gere um artefato
- lide com dependências

PARTE 1 — CONFIGURAÇÃO INICIAL

Execute as células abaixo no Google Colab, na ordem:

Célula 1 — Código do sistema

```
%%writefile calculadora.py  
def somar(a, b):  
    return a + b
```

Célula 2 — Teste

```
%%writefile teste_calculadora.py  
from calculadora import somar  
  
def test_somar():  
    assert somar(2, 2) == 4
```

Célula 3 — Instalar ferramenta de teste

```
!pip install pytest -q
```

Célula 4 — Script de build

```
%%writefile build.py  
import os  
  
print("Iniciando a linha de montagem...")  
  
codigo_resultado = os.system("pytest teste_calculadora.py")
```



```
if codigo_resultado == 0:  
    print("Construção finalizada com sucesso! Software pronto.")  
else:  
    print("A construção parou! Foram encontrados erros nos testes.")
```

Célula 5 — Executar

`!python build.py`

PARTE 2 — ATIVIDADES

A seguir, você irá evoluir o build, adicionando novas responsabilidades.

1. Qualidade do Código

a)

Instale a ferramenta de análise:

```
!pip install flake8 -q
```

- b) Modifique o arquivo `calculadora.py`, criando um problema de qualidade (exemplo: variável não utilizada).
- c) Atualize o `build.py` para incluir a verificação com `flake8`.

2. Geração de Artefato

a)

Modifique o `build.py` para:

- gerar um arquivo `.zip` contendo `calculadora.py`
- realizar essa ação apenas se os testes passarem

Sugestão: utilizar a biblioteca `shutil`.

3. Gerenciamento de Dependências

- a) Adicione uma biblioteca externa no `calculadora.py` (exemplo: `requests`)
- b) Execute o build e observe o comportamento
- c) Crie um arquivo `requirements.txt` contendo a dependência
- d) Instale a dependência com:

```
!pip install -r requirements.txt
```



ENTREGA DA ATIVIDADE

Você deverá entregar no moodle:

Arquivos desenvolvidos

- calculadora.py
- teste_calculadora.py
- build.py
- requirements.txt (quando utilizado)

Evidências da execução

Apresentar prints ou PDF contendo:

- execução do build com sucesso
- execução do build com erro (em pelo menos uma atividade)
- geração do arquivo .zip

Responder às seguintes questões:

- a) O que acontece quando o build encontra um erro?
- b) Qual a importância dos testes no processo?
- c) Para que serve o arquivo requirements.txt?